密码系统设计

第三周预习报告

学习内容

- Head First C 嗨翻 C 语言 第4章
  - 第四章重点：多源文件的编译与链接、头文件与头文件保护、函数声明与定义分离、Makefile 自动化构建
- 《Windows C/C++加密解密实战》第4章
  - 第四章重点：4.2，4.3，4.5（分组密码工作模式、填充机制、OpenSSL EVP 接口调用）
- 课程 mindmap

AI 对学习内容的总结（1分）

要求

让 AI（Kimi，元宝等）阅读学习内容并进行总结。

总结

> **1. C 语言多文件工程（《Head First C》第4章）：** 本章讲解了如何把一个程序拆成多个源文件分别编译，再由链接器合并成可执行文件。核心是头文件的使用：用头文件保护宏（`#ifndef` / `#define` / `#endif`）避免重复包含，用函数声明与定义分离的方式，让调用方只看到接口而不必关心实现细节。章末的 Makefile 则把编译、链接、清理这些命令固化下来，实现一键构建与模块化管理。
>
> **2. 分组密码工作模式与填充机制（《Windows C/C++加密解密实战》第4章）：** 重点 4.2 介绍 ECB、CBC、CFB、OFB、CTR 五种工作模式。ECB 对每个块独立加密，相同明文块产生相同密文；CBC 用前一块密文与当前明文异或，形成密文链；CFB 和 OFB 将块密码转化为流密码；CTR 用计数器生成密钥流。[[修改原因：原文“串成密文链”只对 CBC 准确，ECB 没有链，CFB/OFB/CTR 也不是“链”结构。逐一说明各模式机制更准确。]]重点 4.3 讲填充机制。分组密码要求明文长度为块大小的整数倍，不足时需要按 PKCS#7 标准补齐。PKCS#7 的规则是：假设需要补 b 个字节（1 ≤ b ≤ 块大小），则追加 b 个值为 b 的字节；如果明文已经是整数倍，则额外追加一整块填充。[[修改原因：原文只说“按标准补齐”过于笼统，这里把 PKCS#7 的补齐规则写具体，包括明文已是块大小整数倍时要额外补一整块。]]重点 4.5 讲 OpenSSL 的 EVP 接口调用流

> 程。`EVP_EncryptInit_ex` 初始化上下文并设置密钥和 IV；`EVP_EncryptUpdate` 可多次调用，增量加密数据，输出缓冲区大小应为 `inl + cipher_block_size - 1`；`EVP_EncryptFinal_ex` 处理最后一个不完整块（开启填充时执行 PKCS#7 填充），并结束加密操作。编译时必须链接 `-lcrypto` 库。[[修改原因：补充了三个函数的具体行为和 `EVP_EncryptUpdate` 的缓冲要求，原文只说“分工”太模糊。]]

对 AI 总结的反思与补充（2分）

要求

- AI总结的问题
- 自己的补充

反思与补充

- **AI 总结的不足**：AI 总结把工作模式和填充机制都停在了“流程是什么”，没有说明 CBC 模式下 IV 重复使用会泄露明文前缀的相等关系；也没有把《Head First C》第 4 章讲的链接原理和 `-lcrypto` 的报错联系起来，两部分内容在它眼里是互不相干的。
- **我的补充**：
  - CBC 模式的安全性依赖 IV 每次随机且不可预测。如果 Key 和 IV 的组合在两次加密中重复，会泄露明文第一块的信息，以及两条消息之间任何相同前缀的信息。攻击者可以判断两条消息是否具有相同开头，甚至在选择明文条件下进行猜测攻击。[[修改原因：原文说“判断哪些明文块彼此相同”不够精确——IV 重复主要泄露的是第一块和公共前缀，不是所有块。措辞调整为“Key 和 IV 的组合重复”更准确。]]
  - 这也是 CTR、OFB 只要求 IV 唯一、而不要求 IV 保密的原因。CTR 模式下 IV 复用会导致 trivial decryption，后果比 CBC 更严重。[[修改原因：原文对比对象不清晰。补充 CTR 模式 IV 复用的具体后果，增强对比力度。]]
  - `-lcrypto` 那个 `undefined reference to 'EVP_EncryptInit_ex'` 报错，其实正是《Head First C》第 4 章讲的链接问题：函数只有声明、本地对象文件里没有实现，符号要由外部库提供，而这个符号属于 `libcrypto` 而不属于 `libssl`，漏写链接选项就找不到。教材把“C 语言链接原理”和“OpenSSL 库调用”分开讲，但真写多文件 AES 程序时这两块是撞在一起的。

学习实践过程遇到的问题与解决方式（AI 驱动，优先使用 AI 工具解决问题）（2分）

问题 1：多文件 AES 程序链接 `EVP_EncryptInit_ex` 失败，且明文长度不是整块时输出异常

- **出现环境**：Windows 10 + MinGW (GCC) + OpenSSL 3.x。

- **解决过程**：
  1. 把 AES 加解密拆成 `aes.h`、`aes.c`、`main.c` 三个文件，用 `gcc -c` 分别编译都能通过，但链接成 exe 时报 `undefined reference to 'EVP_EncryptInit_ex'`。
  2. 意识到符号不在本地对象文件里、需要外部库提供，补上 `-lcrypto` 链接选项后通过。
  3. 换 7 字节明文测试时又发现输出长度不对，查资料后发现 `EVP_CIPHER_CTX_set_padding()` 默认是开启的 PKCS#7 填充，会补 n 个值为 n 的字节凑成块大小整数倍。
  4. 调用 `EVP_CIPHER_CTX_set_padding(ctx, 0)` 关闭填充后发现：如果数据长度不是块大小的整数倍，`EVP_EncryptFinal_ex()` 将不会加密剩余数据，并返回错误。也就是说，关闭填充后必须自行保证明文长度对齐。[[修改原因：补充了具体的函数调用形式 `EVP_CIPHER_CTX_set_padding(ctx, 0)` 和“返回错误”这一关键行为。原文只说“不再处理”，没有提到会报错。]]
- **经验总结**：多源文件工程里编译通过不代表链接能过，符号到底来自哪个库要在写 Makefile 时就明确；而 EVP 的填充开关会改变 `EVP_EncryptFinal_ex` 的行为契约，关填充之前必须先确认明文长度。

问题 2：OpenCode 执行依赖安装命令时授权失败，脚本无法运行

- **出现环境**：Windows 10 + OpenCode CLI。
- **解决过程**：
  1. 计划用 OpenCode 智能体自动完成环境安装与导出脚本，但执行到需要授权的命令时中断，提示 `Device approval failed`，后续依赖脚本也被拦截。
  2. 排查发现不是命令本身有问题，而是 OpenCode 的设备授权流程没有走通。
  3. 重新走了一遍设备码/浏览器授权登录流程，授权状态恢复正常。
  4. 授权恢复后再执行同样的依赖安装与脚本命令，正常通过。
- **经验总结**：AI 智能体有明确的权限边界。授权类报错应先确认授权状态，而不是反复改命令。把 AI 用在方案设计、报错分析和代码编写上更可靠；直接让它代办系统级操作（如设备授权、全局安装）收益有限，因为这类操作涉及终端与网页的交互，AI 无法直接完成。[[修改原因：补充了“为什么收益有限”——设备授权需要终端与网页交互，这是 AI 无法直接完成的根本原因。]]

参考资料

AI 工具（你使用的 AI 工具及其链接）

- OpenCode
- Deepseek

图书

- 《Windows C/C++加密解密实战》
- Head First C 嗨翻 C 语言

网站

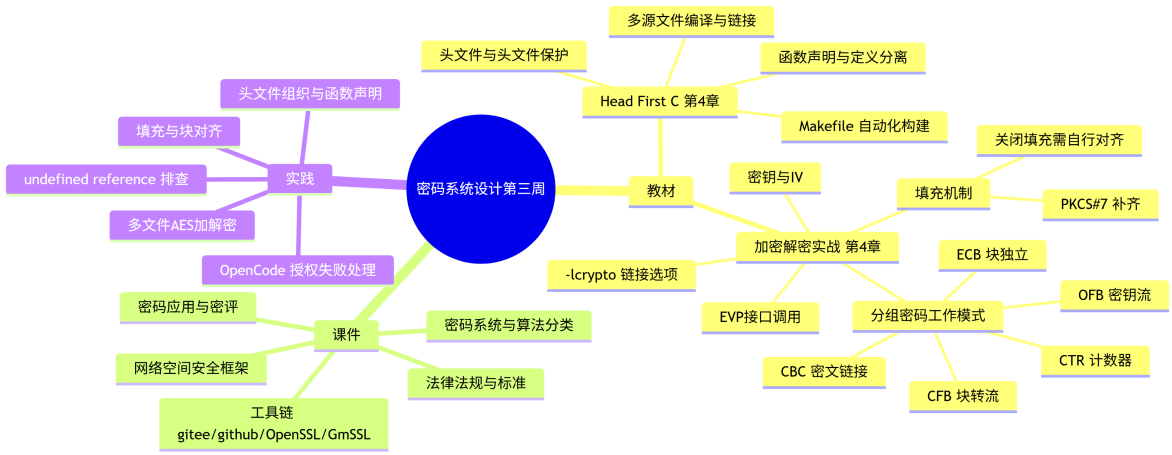
- OpenSSL 官网
- EVP_EncryptInit 官方手册

学习思维导图（2分）

要求

Mermaid 代码与截图，或提交思维导图链接。

思维导图



学习思维导图

```
mindmap
  root((密码系统设计第三周))
    教材
      Head First C 第4章
        多源文件编译与链接
        头文件与头文件保护
        函数声明与定义分离
        Makefile 自动化构建
      加密解密实战 第4章
        分组密码工作模式
          ECB 块独立
```

- CBC 密文链接
- CFB 块转流
- OFB 密钥流
- CTR 计数器

填充机制

- PKCS#7 补齐
- 关闭填充需自行对齐

密钥与IV

EVP接口调用

-lcrypto 链接选项

课件

网络空间安全框架

密码系统与算法分类

法律法规与标准

密码应用与密评

工具链 gitee/github/OpenSSL/GmSSL

实践

多文件AES加解密

头文件组织与函数声明

undefined reference 排查

填充与块对齐

OpenCode 授权失败处理